INSTITUTO POLITECNICO NACIONAL

UNIDAD PROFESIONAL

INTERDICIPLINARIA DE INGENERIA Y CIENCIAS SOCIALES

ADMJNISTRATIVAS

Materia: Ingería de pruebas

Informe de pruebas: Aplicación movil

Equipo de pruebas:

Rodríguez Mendiola Valentina (Analista)

Martínez Lagunas Andrik Jeovany (Programador)

Mejía Ramírez Luis Alejandro (Diseñador)

Jimenez Rogel Sergio (Tester)

Fecha de Elaboración: 19/12/2025

Resumen Ejecutivo

El equipo de pruebas ha concluido la fase de verificación y validación de la aplicación móvil BookSync 1.0. Esta evaluación identifica el estado actual de funcionalidades implementadas, defectos críticos y áreas que requieren desarrollo adicional antes de alcanzar una versión estable.

Estado General:
La aplicación funciona correctamente en sus funciones básicas de gestión de biblioteca personal. Se implementaron las funcionalidades clave como:

- Gestión completa CRUD de libros (crear, editar, eliminar, visualizar)
- Sincronización bidireccional entre SQLite y MySQL mediante API PHP
- Sistema de filtrado por título, autor y género
- Gestión de favoritos con persistencia
- Personalización de colores mediante SharedPreferences
- Pantalla de ayuda informativa

Sin embargo, todavía presenta algunos errores que deben corregirse antes de que pueda considerarse lista para producción.
Se encontraron 3 errores críticos que afectan funciones clave como:

- Sincronización parcial de cambios pendientes (CP-U006, CP-I001)
- Manejo de conflictos en ediciones concurrentes (CP-U004)
- Validación inconsistente de URLs (CP-U011)

Y 4 errores moderados, principalmente relacionados con:

- Rendimiento en sincronización masiva (CP-S003)
- Timeout en conexiones lentas (CP-S004)
- Manejo de caracteres especiales en filtros (casos 34, 47)
- Persistencia de favoritos tras eliminación (caso 28)

No se detectaron errores menores que afecten significativamente el uso de la aplicación. Estos problemas deberán resolverse en las siguientes actualizaciones para garantizar una experiencia completa y estable para los usuarios finales.

Actividades Realizadas

Pruebas de Funcionalidad

- Se validó el proceso completo CRUD de libros (agregar, editar, eliminar, visualizar)
- Se verificó el correcto funcionamiento de la sincronización bidireccional SQLite ↔ MySQL
- Se comprobó el sistema de filtrado combinado (título + autor + género)
- Se validó el sistema de favoritos con persistencia en SharedPreferences
- Se verificó la personalización de colores y su persistencia
- Se comprobó la validación de duplicados por título, autor y fecha
- Se probó el marcado lógico de eliminaciones (eliminado = 1)

Pruebas de Integración

- Se validó la interacción entre módulos Jetpack Compose y SQLiteOpenHelper
- Se verificó comunicación entre DBHelper y archivos PHP (guardar_libro.php, eliminar_libro.php, filtrar_libros.php, sincronizar_mysql_a_sqlite.php)
- Se comprobó el flujo de datos entre pantallas y base de datos local
- Se probó la integración Retrofit/OkHttp con API PHP

Pruebas de Sistema

- Se ejecutaron escenarios end-to-end simulando uso real
- Se validó la estabilidad del sistema con operaciones prolongadas (30 minutos)
- Se verificó el funcionamiento en modo offline/online
- Se comprobó compatibilidad con Android 10, 11 y 12

Pruebas de Seguridad

- Validación de entradas contra inyección SQL
- Se verificó el manejo de errores de red sin exponer información sensible
- Se probó la función arreglarUrl() contra manipulación de rutas
- Validación de IDs para prevenir accesos no autorizados

Pruebas de Rendimiento

- Se midió tiempo de respuesta en operaciones CRUD (< 500ms objetivo)
- Se evaluó sincronización con 50+ registros
- Se verificó uso de memoria durante operación prolongada

Pruebas de Caja Blanca (Cobertura de Código)

- Se validaron que todas las rutas lógicas y condiciones se ejecuten correctamente
- Se analizaron los 5 archivos Kotlin principales (DBHelper.kt, PantallaAgregarLibro.kt, PantallaFiltrarLibros.kt, ColorSettingsScreen.kt, PantallaFavoritos.kt)

- Se identificaron todas las rutas lógicas y se validó su comportamiento ante entradas válidas e inválidas
- Se probaron manejo de excepciones en corutinas y llamadas a red
-

Matriz de Rastreo de Requisitos y Pruebas

ID Req	Descripción del Requisito	Tipo	Casos Asociados	Tipo de Prueba	Estado
REQ-001	Inserción de libros en SQLite	Funcional	CP-U001, CP-U002, CP-U003	Unitaria	PASS
REQ-002	Validación de duplicados	Funcional	CP-U002	Unitaria/Validación	PASS
REQ-003	Edición de libros	Funcional	CP-U004	Unitaria	PARCIAL
REQ-004	Eliminación lógica de libros	Funcional	CP-U005	Unitaria	PASS
REQ-005	Sincronización local → remota	Funcional	CP-U006, CP-I001	Integración	PARCIAL
REQ-006	Sincronización remota → local	Funcional	CP-U007	Unitaria	PASS
REQ-007	Sistema de favoritos	Funcional	CP-U008, CP-I004	Unitaria/Integración	PASS
REQ-008	Filtrado de libros	Funcional	CP-U009	Unitaria	PASS
REQ-009	Personalización de colores	No Funcional	CP-U010, CP-I005	Persistencia	PASS
REQ-010	Validación de URLs	Funcional	CP-U011	Funcional	PARCIAL
REQ-011	Navegación entre pantallas	Funcional	CP-I003, CP-S005	UI/Usabilidad	PASS
REQ-012	Funcionamiento offline	No Funcional	CP-I006, CP-S004	Integración	PASS
REQ-013	Manejo de errores de red	No Funcional	CP-S004	Sistema	PARCIAL
REQ-014	Integridad de datos	No Funcional	CP-S006	Sistema	PASS

ID Req	Descripción del Requisito	Tipo	Casos Asociados	Tipo de Prueba	Estado
REQ-015	Estabilidad general	No Funcional	CP-S001	Sistema	PASS

Casos de Prueba Ejecutados
Pruebas Unitarias – Gestión de Libros

ID	Nombre	Datos	Acción	Resultado Esperado	Tipo	Req.	Resultado
CP-U001	Insertar nuevo libro	Título, autor, fecha, género	Insertar en SQLite	Registro con pendiente_sync=1	Funcional	REQ-001	PASS
CP-U002	Duplicado de libro	Libro existente	libroDuplicado()	Mensaje "Libro ya existente"	Validación	REQ-002	PASS
CP-U003	Campos vacíos	Falta título/autor	Validar formulario	Mensaje "Completa todos los campos"	Límite	REQ-001	PASS
CP-U004	Editar libro	Datos modificados	Guardar cambios	Fecha modificación actualizada	Funcional	REQ-003	PARCIAL
CP-U005	Eliminar libro	ID existente	Marcar eliminado=1	Registro marcado	Funcional	REQ-004	PASS
CP-U006	Sincronización local→remota	pendiente_sync=1	subirCambiosPendientes()	Subido a MySQL	Integración	REQ-005	PARCIAL
CP-U007	Sincronización remota→local	Registros MySQL	sincronizar()	SQLite actualizado	Funcional	REQ-006	PASS
CP-U008	Favoritos	Libro seleccionado	Marcar favorito	Persistencia	Funcional	REQ-007	PASS

ID	Nombre	Datos	Acción	Resultado Esperado	Tipo	Req.	Resultado
				SharedPrefs			
CP-U009	Filtros	Término búsqueda	Aplicar filtros	Lista filtrada	Funcional	REQ-008	PASS
CP-U010	Personalización	Colores elegidos	Guardar SharedPrefs	Tema aplicado	Persistencia	REQ-009	PASS
CP-U011	Arreglo URL	URL relativa	arreglarURL()	URL absoluta	Funcional	REQ-010	PARCIAL

Pruebas de Integración

ID	Caso	Precondición	Pasos	Resultado Esperado	Prioridad	Tipo	Resultado
CP-I001	Sincronización completa	Datos en SQLite y MySQL	1. Modificar libro local 2. Ejecutar sincronización	Datos actualizados sin conflictos	CRÍTICA	Integración	PARCIAL
CP-I002	CRUD ↔ API PHP	Servidor activo	Crear/editar/eliminar libro	Respuesta JSON correcta	ALTA	Integración	PASS
CP-I003	SQLite ↔ Jetpack Compose	App iniciada	Insertar y ver libro en UI	Lista actualizada en tiempo real	ALTA	UI	PASS
CP-I004	Favoritos ↔ Lista principal	Libro seleccionado	Marcar y desmarcar	Vista favoritos correcta	MEDIA	Integración	PASS
CP-I005	Personalización ↔	Preferencias guardada	Reiniciar app	Tema persistente	ALTA	Integración	PASS

ID	Caso	Precondición	Pasos	Resultado Esperado	Prioridad	Tipo	Resultado
	UI	s					
CP-I006	Función offline	Sin internet	Agregar libros	Guardados con pendiente_sync=1	CRÍTICA	Integración	PASS

Pruebas de Sistema

ID	Caso	Precondición	Pasos	Resultado Esperado	Prioridad	Tipo	Resultado
CP-S001	Estabilidad general	App iniciada	Uso 30 minutos	Sin cierres	CRÍTICA	Estabilidad	PASS
CP-S002	Rendimiento	App iniciada	Insertar 20 libros	Respuesta < 500ms	ALTA	Rendimiento	PASS
CP-S003	Sincronización masiva	BD remota 50+ registros	Ejecutar sincronización	Sin bloqueos	ALTA	Rendimiento	PARCIAL
CP-S004	Fallos de red	Modo offline	Intentar sincronizar	Mensaje error adecuado	CRÍTICA	Manejo errores	PARCIAL
CP-S005	Navegación completa	Sistema estable	Navegar todas pantallas	Funcionamiento correcto	MEDIA	Usabilidad	PASS
CP-S006	Integridad datos	Ediciones locales	Sincronizar con remoto	Sin duplicados ni pérdidas	CRÍTICA	Datos	PASS

MATRIZ DE COBERTURA DE PRUEBAS CAJA BLANCA

Archivo	Rutas Totales	Rutas Cubiertas	Cobertura %	Estado
DBHelper.kt	45	38	84%	Buena
PantallaAgregarLibro.kt	12	10	83%	Buena
PantallaFiltrarLibros.kt	18	14	78%	Aceptable
ColorSettingsScreen.kt	8	8	100%	Excelente
PantallaFavoritos.kt	10	9	90%	Excelente
PROMEDIO TOTAL	93	79	85%	Buena

Nota: Cobertura actual 85% — Objetivo ≥ 80% CUMPLIDO

MATRIZ DE CASOS DE PRUEBA DE CAJA BLANCA

ID	Nombre del Caso	Archivo Objetivo	Datos de Prueba	Resultado Esperado	Prioridad	Resultado
CP-CB001	Duplicado con fecha null	DBHelper.kt	titulo="Test", autor="Autor", fecha=null	Validación rechaza entrada	ALTA	PASS
CP-CB002	Sincronizar con JSON malformado	DBHelper.kt	Respuesta API inválida	Exception capturada, log error	CRÍTICA	PASS
CP-CB003	InsertarLibroOffline con ID inválido	DBHelper.kt	ID con < 32 caracteres	Genera MD5 válido	MEDIA	PASS
CP-CB004	ArreglarUrl con null	DBHelper.kt	ruta = null	Retorna null sin crash	ALTA	PASS
CP-CB005	ArreglarUrl con ruta relativa	DBHelper.kt	ruta = "/imagen.jpg"	Retorna BASE_URL + ruta	ALTA	PARCIAL
CP-CB006	SubirCambiosPendientes sin conexión	DBHelper.kt	Timeout en OkHttpClient	Exception manejada	CRÍTICA	PARCIAL
CP-CB007	EnviarLibroSuspend con fecha inválida	DBHelper.kt	fecha = "2024-13-45"	Convierte a "2000-01-01"	MEDIA	PASS
CP-CB008	Agregar libro con URL inválida	PantallaAgregarLibro.kt	urlImagen = "not-a-url"	Mensaje error validación	ALTA	PASS
CP-CB009	Filtro con caracteres	PantallaFiltrarLibros.kt	titulo = "Café & té"	Resultados correctos	MEDIA	PARCIAL

ID	Nombre del Caso	Archivo Objetivo	Datos de Prueba	Resultado Esperado	Prioridad	Resultado
	especiales					
CP-CB010	Favorito de libro eliminado	PantallaFavoritos.kt	Libro marcado + eliminado	No aparece en lista	ALTA	PARCIAL
CP-CB011	Cambiar colores fuera de rango	ColorSettingsScreen.kt	red = 1.5f	Slider limita a 1.0f	BAJA	PASS
CP-CB012	Persistencia colores tras crash	ColorSettingsScreen.kt	Forzar cierre app	SharedPrefs mantiene valores	MEDIA	PASS

RESULTADOS DE COBERTURA ALCANZADA CAJA BLANCA

Archivo	Rutas Totales	Rutas Cubiertas	Cobertura %	Estado	Resultado
DBHelper.kt	45	38	84%	Buena	PASS
PantallaAgregarLibro.kt	12	10	83%	Buena	PASS
PantallaFiltrarLibros.kt	18	14	78%	Aceptable	PARCIAL
ColorSettingsScreen.kt	8	8	100%	Excelente	PASS
PantallaFavoritos.kt	10	9	90%	Excelente	PARCIAL
TOTAL	93	79	85%	Buena	Objetivo alcanzado

Cobertura de Pruebas

- Unitarias: 11 casos → 82% éxito (9 PASS, 2 PARCIAL, 0 FALLO) Verificar individualmente cada componente funcional del sistema
- Integración: 6 casos → 83% éxito (5 PASS, 1 PARCIAL, 0 FALLO) Validar la interacción entre los diferentes módulos
- Sistema: 6 casos → 67% éxito (4 PASS, 2 PARCIAL, 0 FALLO) Evaluar la app como un todo bajo distintos escenarios
- Caja Blanca: 12 casos → 75% éxito (9 PASS, 4 PARCIAL, 0 FALLO) Validar rutas

lógicas y manejo de excepciones interno
- Total: 35 casos técnicos ejecutados

Métricas de Pruebas

1. Cobertura de Pruebas (Test Coverage)
Definición: Mide el porcentaje del código fuente ejecutado por las pruebas.
Fórmula: (Líneas de código ejecutadas / Total líneas de código) × 100%
Resultados:

- DBHelper.kt: 850/1000 líneas = 85%
- PantallaAgregarLibro.kt: 120/150 líneas = 80%
- PantallaFiltrarLibros.kt: 200/250 líneas = 80%
- ColorSettingsScreen.kt: 95/100 líneas = 95%
- HelpScreen.kt: 80/85 líneas = 94%

Cobertura Total del Proyecto: 1345/1585 líneas = 84.9%
Interpretación: La cobertura está por encima del objetivo de 80%, indicando buena cantidad de código probado.

---

2. Tasa de Defectos (Defect Density)
Definición: Número de defectos por cada 1000 líneas de código (KLOC).
Fórmula: Número de defectos / Tamaño del software en KLOC
Resultados:

- Defectos totales encontrados: 9 (0 críticos + 5 moderados + 4 caja blanca)
- Tamaño del código: 1.585 KLOC
- Tasa de defectos: 9 / 1.585 = 5.68 defectos/KLOC

Interpretación: La tasa está dentro del rango aceptable (< 7 defectos/KLOC). Proyectos de alta calidad apuntan a < 3 defectos/KLOC.

3. Tasa de Resolución de Defectos (Defect Fix Rate)
Definición: Velocidad con la que se corrigen defectos después de ser encontrados.
Fórmula: (Defectos corregidos / Defectos encontrados) × 100%
Resultados:

- Defectos encontrados: 9
- Defectos corregidos: 0 (primera fase)
- Tasa de resolución: 0/9 = 0%

Nota: Esta métrica se actualizará en fases posteriores conforme se corrijan los defectos identificados.

---

4. Tasa de Detección de Defectos (Defect Detection Percentage)
Definición: Proporción de defectos detectados durante pruebas vs. total de defectos conocidos.
Fórmula: (Defectos encontrados en pruebas / Total defectos encontrados) × 100%
Resultados:

- Defectos encontrados en pruebas: 9
- Defectos totales conocidos: 9
- Tasa de detección: 9/9 = 100%

Interpretación: Excelente, todos los defectos conocidos fueron detectados durante las pruebas.

---

5. Esfuerzo de Pruebas (Test Effort)
Definición: Tiempo total dedicado a actividades de prueba.
Fórmula: Σ (Horas-persona dedicadas a pruebas)
Resultados:

- Semana 1 (Unitarias): 40 horas
- Semana 2 (Integración): 32 horas
- Semana 3 (Sistema): 28 horas
- Semana 4 (UAT + Caja Blanca): 28 horas
- Esfuerzo total: 128 horas-persona

Distribución por actividad:

- Diseño de casos: 28 horas (21.9%)
- Ejecución: 64 horas (50%)
- Registro de defectos: 16 horas (12.5%)
- Re-pruebas: 20 horas (15.6%)

---

6. Efectividad de las Pruebas (Test Effectiveness)
Fórmula: (Defectos encontrados / Total pruebas ejecutadas) × 100%
Resultados:

- Defectos encontrados: 9
- Pruebas ejecutadas: 35
- Efectividad: 9/35 = 25.7%

Interpretación: Aproximadamente 1 de cada 4 pruebas reveló un defecto. Una tasa de 20-40% es aceptable en estos proyectos.

Resumen de Métricas

Métrica	Valor	Estado	Objetivo
Cobertura de Pruebas	84.9%	✓ Bueno	> 80%
Cobertura Caja Blanca	85%	✓ Bueno	> 80%
Tasa de Defectos	5.68/KLOC	⚠ Aceptable	< 3/KLOC
Tasa de Resolución	0%	Pendiente	> 90%
Detección de Defectos	100%	✓ Excelente	> 85%
Esfuerzo de Pruebas	128 hrs	ℹ Normal	-
Efectividad	25.7%	✓ Bueno	20-40%

Resultados y Conclusiones

La aplicación BookSync 1.0 presenta una arquitectura sólida con separación clara de responsabilidades entre capas de presentación (Jetpack Compose), lógica de negocio (DBHelper) y persistencia (SQLite + MySQL).
Defectos Encontrados:

- Total: 9 (0 críticos, 5 moderados, 4 menores)
- Tasa: 5.68 defectos/KLOC
- Defectos resueltos: en posteriores actualizaciones
- Efectividad de detección: 100%

Recomendaciones

Prioridad Alta
1. Corregir sincronización parcial

2. Resolver conflictos de edición
3. Configurar timeouts adecuados

Agradecimientos
Agradecimiento al equipo de desarrollo y QA por su colaboración en esta fase de pruebas. Esta evaluación ofrece una hoja de ruta clara hacia una versión estable y confiable de BookSync.
El enfoque en métricas y análisis de caja blanca permite una evaluación objetiva del proceso de pruebas y facilita la toma de decisiones informadas para las siguientes etapas del proyecto.